DOCUMENTATION

Astra Gate Portable version from BobH

README

BobH-Astra-Gate-Portable · 2026-11-09

1 这个项目是什么

这是 **Astra Gate**（上游 `239573049/Astra-Gate`）的非官方 Windows 便携版打包仓库。

它不是上游的源码 fork，也不保存上游源码。构建时按上游 release tag 浅克隆源码，把一个可以单独提交给上游的打包脚本叠加进去，再按需应用补丁，构建 Windows 桌面端和内置服务端，最后发布一个免安装的 ZIP。

英文/GitHub 版见 `README.md`；本文件是中文版，`README.pdf` 由 Typst 生成。

> 非官方构建。便携版本身的问题请在本仓库反馈；产品问题请找上游。

2 ZIP 里有什么

- `Astra.exe`：Avalonia 桌面端，Native AOT。
- `resources/server/`：`astra-server.exe`、原生依赖和 Web 控制台。
- 不需要安装程序、管理员权限、Node.js 或 .NET 运行时。

解压后直接运行 `Astra.exe`。默认数据目录仍是上游的 `%USERPROFILE%\.astra`；如果希望数据跟着程序目录走，可以在启动前设置上游支持的 `ASTRA_HOME` 环境变量。

二进制没有代码签名，Windows SmartScreen 可能提示。运行前请核对随附的 SHA-256。

3 仓库结构

只有 `scripts/package-portable.mjs` 使用与上游完全一致的路径；`scripts/` 目录可以 原样复制或合并进上游 checkout，不需要改路径。其余脚本都放在 `wrapper/`，刻意位于 上游目录树之外。

路径	用途	可提交上游？
scripts/package-portable.mjs	在完整上游 checkout 内生成便携 ZIP 的脚本。	是
wrapper/build-portable.mjs	Wrapper 专用编排：解析 tag、克隆、叠加脚本和补丁、构建、打包。	否
wrapper/latest-upstream-tag.mjs	通过 GitHub API 读取最新稳定版 tag。	否
.mise.toml	工具版本和本地任务。	否
.github/workflows/portable-ci.yml	PR/push 构建验证，不发布 Release。	否
.github/workflows/portable-release.yml	定时/手动检查上游、构建并发布 Release。	否
UPSTREAM	一行 `owner/repo`。	否
patches/	可选的源码补丁；真正需要时才创建。	否
README.md, AGENTS.md	文档。	否

4 构建流程

1. 解析上游 tag：显式 `--tag`，或从 GitHub API 取最新稳定版。
2. 浅克隆 `https://github.com/<UPSTREAM>` 到 `.upstream/`（已被 gitignore）。
3. 校验 tag 与 `cli/package.json` 的版本一致，并确认上游仍有本仓库依赖的打包脚本。
4. 把 `scripts/package-portable.mjs` 复制进 checkout。
5. 如果存在 `patches/*.patch`，按文件名顺序应用。
6. 执行与上游 release 相同的构建步骤：pnpm 安装、Web 构建、服务端 Native AOT 发布、
7. 桌面端 Native AOT 构建/发布，然后运行服务端插件自检和桌面端 `--smoke`。
8. 运行 `scripts/package-portable.mjs`：摆放 `Astra.exe` 和服务端，生成 ZIP 与 SHA-256，
9. 解压 ZIP 并校验布局与文件大小。
10. 写入 `build-info.json`（上游 tag、RID、校验和、大小、workflow run）。

产物位于 `out/portable/<rid>/`：

```
Astra-<version>-win32-<arch>-portable.zip
Astra-<version>-win32-<arch>-portable.zip.sha256
build-info.json
```

5 本地构建

需要：

- Windows 10/11，x64 或 arm64。
- Git、PowerShell。
- Node.js 24、pnpm 9.15.0。
- .NET SDK `11.0.100-rc.3.26522.148`（上游 `global.json` 固定的版本）。

使用 mise（`mise install` 只安装 Node.js 和 pnpm；.NET SDK 需单独安装）：

```
mise install
mise run portable -- --tag v0.14.3100
# 最新稳定版
mise run portable
```

不使用 mise：

```
node wrapper/build-portable.mjs --tag v0.14.3100 --rid win-x64
node wrapper/build-portable.mjs --tag v0.14.3100 --rid win-arm64
```

常用检查：

```
# 只克隆、叠加、校验，不执行重构建
node wrapper/build-portable.mjs --tag v0.14.3100 --prepare-only

# 查看最新稳定版 tag
node wrapper/latest-upstream-tag.mjs

# 校验 mise 任务和脚本语法
mise tasks validate
node --check wrapper/build-portable.mjs
node --check wrapper/latest-upstream-tag.mjs
node --check scripts/package-portable.mjs
```

`--prepare-only` 可以在非 Windows 主机上运行；完整构建需要 Windows，因为 Windows Native AOT 发布和 ZIP 步骤依赖 Windows 工具链。

6 CI 与自动发布

`Portable CI` 在 PR 和 `master` push 且命中 wrapper 文件时运行，构建最新稳定版并上传 workflow artifact，不创建 Release。

`Portable release` 有两种触发方式：

- 每 6 小时定时运行。
- 手动 `workflow_dispatch`，可填写 `tag` 、 `rid` 和 `force` 。

定时任务会解析最新稳定版 tag，检查本仓库是否已有 `portable-<tag>` Release；不存在 才构建。手动可以强制重建。Release tag：

```
portable-<tag>    # 例如 portable-v0.14.3100
```

附上 ZIP、 `.sha256` 和 `build-info.json` 。预览版会标记为 prerelease。定时构建默认 `win-x64` ； `win-arm64` 需要手动触发，并依赖 `windows-11-arm` runner。

6.1 Scoop 与校验和

每个 Release 都上传 `Astra-<version>-win32-<arch>-portable.zip.sha256` ，格式为标准 `<sha256>  <filename>` 。release workflow 在发布前用 `sha256sum -c` 校验，并把 hash 写进 Release notes。Scoop manifest 可以把 autoupdate 的 hash 指向该文件：

```
"hash": {
  "url": "$url.sha256"
}
```

因为 release asset 就是 `<zip>.sha256` 。ARM 构建按需调整 manifest 的 URL。

6.2 为什么不用 `on: release`

GitHub Actions 的 `release` 事件只在发布 Release 的那个仓库里触发。Wrapper 仓库（或 fork）无法订阅上游仓库的 Release 事件。可选方案：

1. 定时轮询（当前实现）：不需要上游配合，最多约 6 小时延迟；GitHub 在高峰期可能 会推迟定时任务。
2. 上游 **`repository_dispatch`** ：需要上游维护者加 token 和 dispatch 步骤；如果对方 同意，这是延迟最低的方案。
3. **Webhook / GitHub App / RSS** 桥接：需要额外服务托管和维护。

定时 workflow 只在默认分支上运行。请把 `master` 推到 GitHub 并设为默认分支，自动发布 才会生效。

7 PR 部分和 wrapper 部分怎么分开

GitHub 没有“某路径只能出现在 PR 里”的配置文件。PR 的内容就是 fork 中某个分支上的 commit；`CODEOWNERS` 、路径过滤和必需检查只影响 review 和 CI，不会限制 PR 改了哪些 文件。

所以边界必须靠结构来保证：

- 本仓库保存所有 wrapper-only 文件：workflow、mise 配置、编排脚本和文档。
- 要向上游提 PR 时，fork Astra Gate，从上游 `main` 开分支，只复制 `scripts/package-portable.mjs` 这一个文件，提交并开 PR。
- fork 的默认分支保持与上游一致；不要把 wrapper workflow 或 mise 配置放上去。
- 即使在本仓库，也把“可 PR 的改动”和“wrapper 改动”分成不同 commit。混在一起时， 导出 PR 分支前先拆开。

开 PR 前先开 issue 问上游：便携 ZIP 是否受欢迎、脚本应放在哪个路径。打包脚本是刻意 做成自包含的，只使用上游已有的 `scripts/rid-map.mjs` 和 `scripts/pack-platform.mjs` ， 不依赖本仓库。

8 补丁

不要预先创建 `patches/` 。确实需要改源码时：

1. 添加 `patches/NNNN-short-description.patch` 。
2. 编排脚本会在复制打包脚本后，按文件名顺序应用补丁。
3. 每个补丁先做 `git apply --check` ；补丁不再适用时构建直接失败。

补丁不会自动 rebase。定时构建失败就是需要维护补丁的信号；这比静默自动合并源码补丁 更安全。

9 发布与版本

- Release tag： `portable-<tag>` ，例如 `portable-v0.14.3100` 。
- 资产： `Astra-<version>-win32-x64-portable.zip` 、 `.sha256` 、 `build-info.json` 。
- 预览版（版本号含 `-preview.N` ）标记为 prerelease。

- Release 由 workflow 使用 `GITHUB_TOKEN` 创建；仓库的 Actions 权限需要允许 `contents: write`（workflow 已在 release job 里声明）。
- 定时任务只在默认分支运行；先把 `master` 设为默认分支。

10 限制

- 仅 Windows；这里没有 macOS/Linux 便携构建。
- 没有安装器集成、自动更新集成或代码签名。
- 数据目录默认不迁移；需要自包含目录时使用 `ASTRA_HOME`。
- 自动发现只跟踪稳定版；预览版需要手动填 tag。
- GitHub 可能在仓库 60 天无活动后停用定时 workflow；如果定时任务停了，到 Actions 页面 重新启用。
- 构建较重（大约 30–60 分钟），因为要构建两个 Native AOT 二进制。